

ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH  
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN  
KHOA MẠNG MÁY TÍNH VÀ TRUYỀN THÔNG

BÙI NGUYỄN TƯỜNG VÂN - 23521768

ĐỒ ÁN CHUYÊN NGÀNH  
THIẾT KẾ VÀ XÂY DỰNG HỆ THỐNG BẢNG  
TIN ĐIỆN TỬ DỰA TRÊN NỀN TẢNG IOT ĐIỀU  
KHIỂN QUA ỨNG DỤNG DI ĐỘNG

**Design and Implementation of an IoT-based  
Electronic Bulletin Board Controlled via Mobile  
Application**

GIẢNG VIÊN HƯỚNG DẪN

ThS. LÊ ANH TUẤN

TP. HỒ CHÍ MINH, 2026

## LỜI CẢM ƠN

Em xin gửi lời cảm ơn chân thành đến thầy Lê Anh Tuấn – giảng viên bộ môn “Đồ án chuyên ngành” thuộc Khoa Mạng Máy tính và Truyền thông, Trường Đại học Công nghệ Thông tin – ĐHQG-HCM. Trong suốt quá trình thực hiện đề tài, thầy đã tận tình hướng dẫn, hỗ trợ và truyền đạt cho em những kiến thức cũng như kinh nghiệm quý báu để em có thể hoàn thành đề tài này.

Tuy đã cố gắng nghiên cứu và thực hiện đề tài một cách nghiêm túc, nhưng do kiến thức chuyên môn và kinh nghiệm thực tế của em còn hạn chế nên bài làm khó tránh khỏi những thiếu sót trong quá trình tìm hiểu, đánh giá và trình bày nội dung. Em rất mong nhận được những ý kiến đóng góp từ thầy để đề tài được hoàn thiện hơn.

Em xin chân thành cảm ơn.

## MỤC LỤC

|                  |                                                |           |
|------------------|------------------------------------------------|-----------|
| <b>CHƯƠNG 1.</b> | <b>TỔNG QUAN ĐỀ TÀI</b>                        | <b>6</b>  |
| 1.1.             | Giới thiệu đề tài                              | 6         |
| 1.2.             | Lý do chọn đề tài                              | 6         |
| 1.3.             | Ý nghĩa thực tiễn của đề tài                   | 7         |
| <b>CHƯƠNG 2.</b> | <b>CƠ SỞ LÝ THUYẾT</b>                         | <b>8</b>  |
| 2.1.             | Tổng quan về LED Matrix                        | 8         |
| 2.1.1.           | Nguyên lý hoạt động của LED Matrix             | 8         |
| 2.2.             | Tổng quan về ESP8266                           | 8         |
| 2.2.1.           | Nguyên lý hoạt động của ESP8266                | 9         |
| 2.2.2.           | Web Server trên ESP8266                        | 9         |
| 2.3.             | Tổng quan về Flutter                           | 9         |
| 2.3.1.           | Kiến trúc Flutter                              | 9         |
| 2.4.             | Giao thức HTTP trong hệ thống điều khiển IoT   | 10        |
| 2.4.1.           | Truyền tham số điều khiển qua URL              | 10        |
| 2.5.             | Tổng quan hệ thống điều khiển LED qua WiFi     | 11        |
| 2.5.1.           | Mô hình hoạt động                              | 11        |
| 2.5.2.           | Luồng dữ liệu hệ thống                         | 11        |
| <b>CHƯƠNG 3.</b> | <b>PHÂN TÍCH THIẾT KẾ HỆ THỐNG</b>             | <b>12</b> |
| 3.1.             | Kiến trúc tổng thể hệ thống                    | 12        |
| 3.2.             | Thiết kế phần cứng                             | 12        |
| 3.2.1.           | Kết nối ESP8266 với LED Matrix                 | 12        |
| 3.2.2.           | Sơ đồ kết nối phần cứng                        | 13        |
| 3.3.             | Thiết kế phần mềm                              | 13        |
| 3.3.1.           | Thiết kế chương trình ESP8266                  | 14        |
| 3.3.2.           | Thiết kế ứng dụng Flutter                      | 15        |
| 3.4.             | Thiết kế giao tiếp giữa Flutter App và ESP8266 | 15        |
| 3.4.1.           | Luồng gửi dữ liệu                              | 16        |
| 3.4.2.           | Thiết kế URI điều khiển                        | 17        |
| 3.5.             | Thiết kế giao diện người dùng                  | 18        |

|                                               |                                                |           |
|-----------------------------------------------|------------------------------------------------|-----------|
| 3.5.1.                                        | Các thành phần giao diện chính                 | 19        |
| <b>CHƯƠNG 4. HIỆN THỰC HỆ THỐNG</b>           |                                                | <b>23</b> |
| 4.1.                                          | Cài đặt môi trường phát triển                  | 23        |
| 4.1.2.                                        | Cài đặt Flutter SDK và Android Studio          | 23        |
| 4.2.                                          | Lập trình ESP8266                              | 23        |
| 4.3.                                          | Hiệu ứng và chức năng hiển thị trên LED Matrix | 28        |
| 4.3.1.                                        | Xử lý văn bản hiển thị                         | 28        |
| 4.3.2.                                        | Điều chỉnh tốc độ chạy chữ                     | 29        |
| 4.3.3.                                        | Điều chỉnh độ sáng LED Matrix                  | 29        |
| 4.3.4.                                        | Thiết lập số vòng lặp chữ                      | 30        |
| 4.3.5.                                        | Hiệu ứng hiện từng chữ                         | 30        |
| 4.3.6.                                        | Hiệu ứng nhấp nháy (Blink)                     | 32        |
| 4.3.7.                                        | Lệnh dừng hiển thị (STOP RUNNING)              | 32        |
| 4.3.8.                                        | Hiển thị thời gian thực                        | 33        |
| 4.3.9.                                        | Chức năng xác thực người dùng                  | 33        |
| 4.3.10.                                       | Chức năng hẹn giờ tự động                      | 35        |
| 4.3.11.                                       | Chế độ Emergency Mode                          | 36        |
| 4.4.                                          | Triển khai và kiểm thử hệ thống                | 36        |
| 4.4.1.                                        | Kiểm thử kết nối WiFi                          | 37        |
| 4.4.2.                                        | Kiểm thử hiển thị LED Matrix                   | 37        |
| 4.4.3.                                        | Kiểm thử ứng dụng Flutter                      | 38        |
| <b>CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN</b> |                                                | <b>39</b> |
| 5.1.                                          | Kết luận                                       | 39        |
| 5.2.                                          | Hướng phát triển                               | 39        |
| <b>TÀI LIỆU THAM KHẢO</b>                     |                                                | <b>40</b> |

## CHƯƠNG 1. TỔNG QUAN ĐỀ TÀI

### 1.1. Giới thiệu đề tài

Trong thời đại chuyển đổi số hiện nay, Internet of Things (IoT) đang được ứng dụng rộng rãi trong nhiều lĩnh vực như nhà thông minh, giám sát môi trường, tự động hóa và điều khiển thiết bị từ xa. Nhờ khả năng kết nối Internet và trao đổi dữ liệu theo thời gian thực, các hệ thống IoT giúp nâng cao tính linh hoạt, tiết kiệm thời gian và tối ưu hóa quá trình quản lý thiết bị.

Bên cạnh đó, nhu cầu hiển thị thông tin nhanh chóng và trực quan thông qua các bảng tin điện tử ngày càng phổ biến tại trường học, cửa hàng, quán cà phê, nhà ở hoặc các khu vực công cộng. Tuy nhiên, các bảng LED truyền thống thường có khả năng điều khiển hạn chế, phải cập nhật nội dung thủ công hoặc phụ thuộc vào máy tính kết nối trực tiếp, gây bất tiện trong quá trình sử dụng.

Xuất phát từ thực tế đó, đề tài “Thiết kế và xây dựng hệ thống bảng tin điện tử dựa trên nền tảng IoT điều khiển qua ứng dụng di động” được thực hiện nhằm xây dựng một hệ thống bảng tin LED có khả năng kết nối WiFi, cho phép người dùng điều khiển và cập nhật nội dung hiển thị thông qua ứng dụng di động Flutter. Hệ thống sử dụng vi điều khiển ESP8266 kết hợp với LED Matrix MAX7219 để hiển thị thông tin theo thời gian thực.

Ngoài các chức năng hiển thị văn bản cơ bản, hệ thống còn hỗ trợ nhiều tính năng như điều chỉnh tốc độ chạy chữ, độ sáng LED, hiệu ứng hiển thị, hiển thị thời gian thực, hẹn giờ tự động và chế độ khẩn cấp (Emergency Mode). Qua đó, đề tài hướng đến việc xây dựng một mô hình điều khiển LED thông minh, trực quan, dễ sử dụng và có khả năng mở rộng trong các ứng dụng IoT thực tế.

### 1.2. Lý do chọn đề tài

Hiện nay, các hệ thống bảng tin điện tử được sử dụng ngày càng nhiều để hiển thị thông báo, quảng cáo hoặc truyền tải thông tin trong thời gian thực. Tuy nhiên, phần lớn các hệ thống truyền thống vẫn còn tồn tại nhiều hạn chế như khó cập nhật nội dung từ xa, thiếu tính linh hoạt và giao diện điều khiển chưa thân thiện với người dùng.

Trong khi đó, sự phát triển mạnh mẽ của công nghệ IoT và điện thoại thông minh đã mở ra khả năng điều khiển thiết bị từ xa thông qua kết nối mạng không dây. Việc kết hợp giữa IoT và ứng dụng di động giúp hệ thống trở nên thông minh hơn, thuận tiện hơn và dễ dàng quản lý ở bất kỳ đâu có kết nối Internet hoặc WiFi nội bộ.

Ngoài ra, vi điều khiển ESP8266 có chi phí thấp nhưng tích hợp sẵn khả năng kết nối WiFi, phù hợp cho các ứng dụng IoT nhỏ gọn. Kết hợp với LED Matrix MAX7219 và nền tảng Flutter, hệ thống có thể xây dựng được một giao diện điều khiển trực quan, hiện đại và dễ mở rộng thêm nhiều chức năng trong tương lai.

Thông qua quá trình thực hiện đề tài, em mong muốn vận dụng và củng cố các kiến thức về lập trình nhúng, truyền thông mạng không dây, giao tiếp HTTP, thiết

kế ứng dụng di động và xây dựng hệ thống IoT thực tế. Đồng thời, đề tài cũng góp phần tạo ra một mô hình bảng tin điện tử thông minh có tính ứng dụng cao trong học tập và đời sống.

### **1.3. Ý nghĩa thực tiễn của đề tài**

Đề tài “Thiết kế và xây dựng hệ thống bảng tin điện tử dựa trên nền tảng IoT điều khiển qua ứng dụng di động” mang ý nghĩa thực tiễn cao trong việc ứng dụng công nghệ IoT vào các hệ thống hiển thị thông tin thông minh. Hệ thống cho phép người dùng dễ dàng cập nhật và điều khiển nội dung hiển thị từ xa thông qua ứng dụng di động mà không cần thao tác trực tiếp trên phần cứng, góp phần nâng cao tính linh hoạt và thuận tiện trong quá trình sử dụng.

Bên cạnh đó, hệ thống có thể được ứng dụng trong nhiều môi trường thực tế như trường học, cửa hàng, quán cà phê, văn phòng hoặc các khu vực công cộng để hiển thị thông báo, quảng cáo hoặc thông tin thời gian thực. Các chức năng như điều chỉnh tốc độ chạy chữ, độ sáng, hiệu ứng hiển thị, hẹn giờ tự động và chế độ khẩn cấp giúp hệ thống đáp ứng tốt hơn nhu cầu sử dụng thực tế của người dùng.

Ngoài giá trị ứng dụng, đề tài còn giúp củng cố và mở rộng kiến thức về lập trình nhúng, mạng không dây, giao tiếp HTTP, phát triển ứng dụng di động Flutter và xây dựng hệ thống IoT hoàn chỉnh. Đây cũng là nền tảng để tiếp tục nghiên cứu và phát triển các hệ thống điều khiển thông minh có khả năng mở rộng trong tương lai như nhà thông minh (Smart Home), hệ thống cảnh báo tự động hoặc bảng thông tin điện tử kết nối Internet.

## CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

### 2.1. Tổng quan về LED Matrix



LED Matrix là một dạng ma trận đèn LED được sắp xếp theo hàng và cột nhằm hiển thị ký tự, văn bản hoặc các hiệu ứng đơn giản. Trong đề tài này, em sử dụng LED Matrix 32x8 tích hợp chip điều khiển MAX7219 để thực hiện chức năng hiển thị chữ chạy. Module bao gồm 4 ma trận LED 8x8 ghép lại với nhau, cho phép hiển thị nội dung dạng cuộn theo chiều ngang.

LED Matrix được sử dụng phổ biến trong các bảng thông báo điện tử, bảng quảng cáo mini, đồng hồ điện tử và các hệ thống hiển thị thông tin do có ưu điểm nhỏ gọn, tiết kiệm điện năng và dễ lập trình điều khiển. Trong hệ thống của đề tài, LED Matrix đóng vai trò là thiết bị đầu ra, nhận dữ liệu từ ESP8266 để hiển thị nội dung do người dùng gửi từ ứng dụng di động.

#### 2.1.1. Nguyên lý hoạt động của LED Matrix

LED Matrix hoạt động dựa trên nguyên lý quét hàng và cột. Vi điều khiển sẽ gửi dữ liệu điều khiển đến chip MAX7219 thông qua giao tiếp SPI. Sau đó, chip MAX7219 thực hiện việc bật/tắt các LED tương ứng để tạo thành ký tự hoặc hiệu ứng hiển thị.

Trong hệ thống của đề tài, ứng dụng Flutter gửi nội dung văn bản đến ESP8266 thông qua giao thức HTTP qua mạng WiFi. ESP8266 tiếp nhận và xử lý chuỗi dữ liệu, sau đó truyền dữ liệu hiển thị đến LED Matrix. Các ký tự được chuyển đổi thành dạng bitmap và hiển thị theo hiệu ứng chạy chữ từ phải sang trái trên ma trận LED.

### 2.2. Tổng quan về ESP8266



ESP8266 là dòng vi điều khiển tích hợp WiFi được sử dụng phổ biến trong các hệ thống Internet of Things (IoT). Thiết bị có khả năng kết nối mạng không dây, xử

lý dữ liệu và giao tiếp với các thiết bị phần cứng khác thông qua nhiều chuẩn giao tiếp khác nhau như SPI, UART và I2C.

Trong đề tài này, ESP8266 NodeMCU đóng vai trò là bộ điều khiển trung tâm của hệ thống. Thiết bị có nhiệm vụ nhận dữ liệu từ ứng dụng Flutter thông qua mạng WiFi, xử lý các yêu cầu điều khiển và truyền dữ liệu hiển thị đến LED Matrix MAX7219.

### 2.2.1. Nguyên lý hoạt động của ESP8266

ESP8266 hoạt động bằng cách kết nối vào mạng WiFi nội bộ và mở một Web Server tại cổng 80 để tiếp nhận các yêu cầu điều khiển từ ứng dụng Flutter. Khi người dùng gửi dữ liệu từ ứng dụng di động, ESP8266 sẽ tiếp nhận các tham số như nội dung văn bản, tốc độ chạy chữ, độ sáng hoặc chế độ nhấp nháy thông qua giao thức HTTP.

Sau khi xử lý dữ liệu nhận được, ESP8266 truyền dữ liệu điều khiển đến LED Matrix MAX7219 thông qua giao tiếp SPI để thực hiện hiển thị nội dung trên ma trận LED. Toàn bộ hệ thống hoạt động theo mô hình client-server, trong đó Flutter App đóng vai trò client và ESP8266 đóng vai trò server xử lý dữ liệu.

### 2.2.2. Web Server trên ESP8266

Trong hệ thống, ESP8266 được cấu hình để hoạt động như một Web Server sử dụng lớp WiFiServer trên cổng 80. Web Server này có nhiệm vụ lắng nghe các kết nối từ ứng dụng Flutter và xử lý các HTTP Request gửi đến từ người dùng.

Sau khi kết nối WiFi thành công, ESP8266 khởi động server và chờ client gửi yêu cầu điều khiển. Khi có request được gửi tới, ESP8266 sẽ đọc nội dung URL, phân tích các tham số như text, speed, brightness, loops, blink hoặc stop, sau đó thực hiện hành động tương ứng trên LED Matrix. Ngoài ra, ESP8266 còn có thể phản hồi lại dữ liệu HTML cho client nhằm kiểm tra trạng thái hoạt động của hệ thống.

## 2.3. Tổng quan về Flutter



Flutter là framework phát triển ứng dụng đa nền tảng do Google phát triển, sử dụng ngôn ngữ lập trình Dart. Flutter cho phép xây dựng ứng dụng cho Android và iOS từ một mã nguồn duy nhất với giao diện trực quan và hiệu năng cao.

Trong đề tài này, Flutter được sử dụng để xây dựng ứng dụng điều khiển LED Matrix thông qua kết nối WiFi với ESP8266. Ứng dụng cho phép người dùng nhập nội dung hiển thị, điều chỉnh tốc độ chạy chữ, độ sáng, số lần lặp và các hiệu ứng hiển thị khác.

### 2.3.1. Kiến trúc Flutter

Kiến trúc của Flutter được xây dựng theo dạng widget-based, trong đó toàn bộ giao diện được cấu thành từ các widget khác nhau. Các widget này có thể kết hợp

để tạo nên giao diện người dùng hoàn chỉnh và dễ dàng mở rộng chức năng.

Trong ứng dụng của đề tài, giao diện Flutter bao gồm nhiều thành phần như TextField để nhập văn bản, tốc độ và độ sáng; SwitchListTile để bật/tắt chế độ nhấp nháy; cùng các ElevatedButton để gửi dữ liệu hoặc dừng hiển thị trên LED Matrix. Dữ liệu từ các widget sẽ được xử lý và chuyển thành HTTP Request gửi đến ESP8266.

## 2.4. Giao thức HTTP trong hệ thống điều khiển IoT

Trong hệ thống điều khiển LED Matrix qua WiFi, giao thức HTTP được sử dụng làm phương thức truyền dữ liệu giữa ứng dụng Flutter và ESP8266. Ứng dụng Flutter đóng vai trò là client, còn ESP8266 đóng vai trò là Web Server nhận và xử lý yêu cầu điều khiển.

Khi người dùng nhập nội dung hoặc thay đổi các thông số trên ứng dụng, Flutter sẽ tạo một HTTP request chứa các tham số điều khiển và gửi đến địa chỉ IP của ESP8266. Sau khi nhận được request, ESP8266 phân tích dữ liệu, cập nhật trạng thái hệ thống và điều khiển LED Matrix hiển thị nội dung tương ứng.

### 2.4.1. Truyền tham số điều khiển qua URL

Việc truyền tham số điều khiển trong hệ thống được thực hiện thông qua query parameters trên URL. Khi người dùng nhấn nút gửi dữ liệu, ứng dụng Flutter sẽ tạo URL theo dạng:

`http://<địa_chỉ_IP_ESP8266>/?text=<nội_dung>&speed=<tốc_độ>&brightness=<độ_sáng>&loops=<số_lần_lặp>&effect=<hiệu_ứng>&blink=1&stop=1&end=1`

Trong đó, các tham số chính gồm:

|                                     |   |
|-------------------------------------|---|
| +                                   | t |
| ext: nội dung văn bản cần hiển thị  |   |
| +                                   | s |
| peed: tốc độ chạy chữ               |   |
| +                                   | b |
| rightness: độ sáng LED Matrix       |   |
| +                                   | l |
| oops: số vòng lặp hiển thị          |   |
| +                                   | e |
| ffect: hiệu ứng hiện từng chữ       |   |
| +                                   | b |
| link: bật hiệu ứng nhấp nháy        |   |
| +                                   | s |
| top: dừng hiển thị                  |   |
| +                                   | e |
| nd: đánh dấu kết thúc chuỗi tham số |   |

Trước khi gửi, nội dung văn bản được xử lý để loại bỏ dấu tiếng Việt và các ký tự không phù hợp. Sau đó, chuỗi được mã hóa bằng `Uri.encodeComponent()` nhằm đảm bảo dữ liệu truyền qua URL không bị lỗi. Cách truyền dữ liệu này đơn giản, dễ triển khai và phù hợp với hệ thống IoT quy mô nhỏ trong mạng LAN.

## 2.5. Tổng quan hệ thống điều khiển LED qua WiFi

Hệ thống điều khiển LED qua WiFi gồm ba thành phần chính: ứng dụng Flutter, ESP8266 NodeMCU và LED Matrix 32x8 sử dụng chip MAX7219. Trong đó, ứng dụng Flutter là giao diện điều khiển, ESP8266 là bộ xử lý trung tâm, còn LED Matrix là thiết bị đầu ra dùng để hiển thị nội dung.

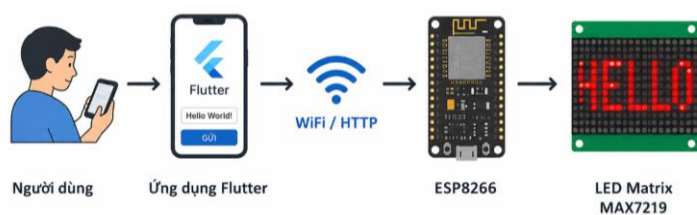
Các thiết bị hoạt động trong cùng một mạng WiFi. Người dùng nhập dữ liệu trên ứng dụng di động, sau đó ứng dụng gửi yêu cầu điều khiển đến địa chỉ IP của ESP8266. ESP8266 tiếp nhận dữ liệu, xử lý thông tin và truyền tín hiệu điều khiển đến LED Matrix để hiển thị chữ chạy, hiệu ứng nhấp nháy, giờ hiện tại hoặc chế độ khẩn cấp.

### 2.5.1. Mô hình hoạt động

Mô hình hoạt động của hệ thống được xây dựng theo dạng client-server. Ứng dụng Flutter trên điện thoại đóng vai trò client gửi yêu cầu điều khiển. ESP8266 đóng vai trò server, tiếp nhận và xử lý request từ ứng dụng. LED Matrix nhận dữ liệu điều khiển từ ESP8266 thông qua giao tiếp phần cứng để hiển thị nội dung.

Luồng hoạt động tổng quát của hệ thống như sau:

**Người dùng → Ứng dụng Flutter → WiFi/HTTP → ESP8266 → LED Matrix MAX7219**



Hình 1. Luồng hoạt động tổng quát

Với mô hình này, người dùng có thể điều khiển nội dung hiển thị trên LED Matrix trực tiếp từ điện thoại mà không cần thao tác thủ công trên phần cứng.

### 2.5.2. Luồng dữ liệu hệ thống

Luồng dữ liệu của hệ thống bắt đầu khi người dùng nhập địa chỉ IP của ESP8266 và nội dung cần hiển thị trên ứng dụng Flutter. Người dùng có thể điều chỉnh thêm tốc độ chạy chữ, độ sáng, số vòng lặp, hiệu ứng hiện từng chữ, chế độ nhấp nháy hoặc lệnh dừng.

Sau khi nhấn nút gửi, Flutter xử lý nội dung đầu vào, loại bỏ dấu tiếng Việt, mã hóa chuỗi văn bản và tạo HTTP GET request. Request này được gửi đến ESP8266 thông qua mạng WiFi. ESP8266 nhận request, phân tích các tham số trong URL và cập nhật trạng thái điều khiển tương ứng.

Cuối cùng, ESP8266 truyền dữ liệu đã xử lý đến LED Matrix để hiển thị. Ngoài chức năng gửi nội dung thông thường, hệ thống còn hỗ trợ hiển thị giờ hiện tại, hẹn giờ tự động gửi nội dung và chế độ Emergency Mode hiển thị cảnh báo SOS với độ sáng cao và hiệu ứng nhấp nháy.

## CHƯƠNG 3. PHÂN TÍCH THIẾT KẾ HỆ THỐNG

### 3.1. Kiến trúc tổng thể hệ thống

Hệ thống được thiết kế theo mô hình điều khiển từ xa dựa trên nền tảng IoT, bao gồm ba thành phần chính:

- + **Ứng dụng Flutter trên thiết bị di động:** Ứng dụng Flutter đóng vai trò là giao diện điều khiển người dùng, cho phép nhập nội dung hiển thị và thiết lập các tham số điều khiển như tốc độ chạy chữ, độ sáng, số vòng lặp, hiệu ứng nhấp nháy, hiệu ứng hiện từng chữ, hẹn giờ hiển thị và chế độ khẩn cấp.
- + **Vi điều khiển ESP8266 nodemcu:** ESP8266 NodeMCU là bộ điều khiển trung tâm của hệ thống. Thiết bị này kết nối vào mạng WiFi, đóng vai trò như một Web Server để nhận các yêu cầu HTTP từ ứng dụng Flutter. Sau khi nhận dữ liệu, ESP8266 tiến hành phân tích các tham số được truyền qua URL, xử lý nội dung văn bản và điều khiển LED Matrix hiển thị theo cấu hình tương ứng.
- + **Module LED Matrix 32x8 sử dụng chip MAX7219:** LED Matrix 32x8 sử dụng chip MAX7219 là thành phần hiển thị của hệ thống. Module này giao tiếp với ESP8266 thông qua chuẩn SPI, có nhiệm vụ hiển thị văn bản dạng chạy ngang, nhấp nháy hoặc hiện từng chữ tùy theo chế độ được người dùng lựa chọn.

Về luồng hoạt động, người dùng thao tác trên ứng dụng Flutter, sau đó ứng dụng tạo HTTP request chứa nội dung và các tham số điều khiển gửi đến địa chỉ IP của ESP8266. ESP8266 tiếp nhận request, xử lý dữ liệu và gửi tín hiệu điều khiển đến LED Matrix. Nhờ đó, nội dung hiển thị trên bảng LED có thể được cập nhật nhanh chóng, linh hoạt và không cần thao tác trực tiếp trên phần cứng.

### 3.2. Thiết kế phần cứng

Phần cứng của hệ thống được xây dựng dựa trên vi điều khiển ESP8266 NodeMCU và module LED Matrix 32x8 sử dụng chip điều khiển MAX7219. ESP8266 đóng vai trò là bộ điều khiển trung tâm, đảm nhiệm việc kết nối WiFi, tiếp nhận dữ liệu từ ứng dụng Flutter và truyền tín hiệu điều khiển đến LED Matrix để hiển thị nội dung.

Hệ thống được thiết kế theo hướng nhỏ gọn, dễ triển khai và có khả năng mở rộng cho các ứng dụng IoT thực tế. Các thành phần phần cứng chính bao gồm:

- + ESP8266 NodeMCU.
- + Module LED Matrix MAX7219 32x8.
- + Nguồn cấp 5V.
- + Dây kết nối tín hiệu SPI.

#### 3.2.1. Kết nối ESP8266 với LED Matrix

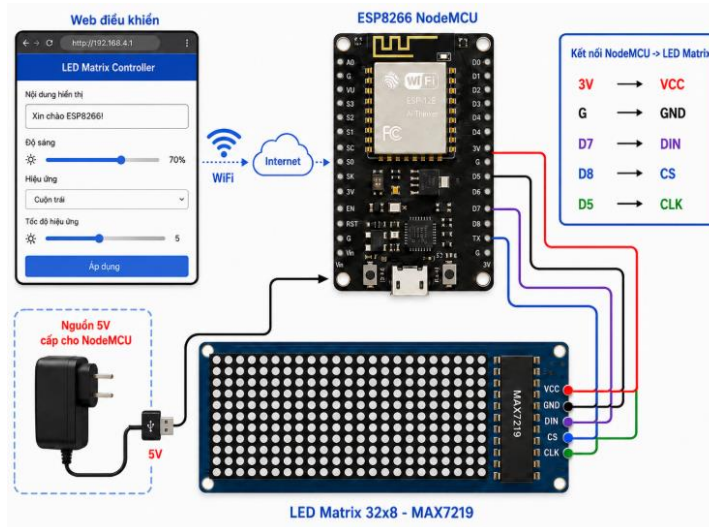
ESP8266 NodeMCU giao tiếp với LED Matrix MAX7219 thông qua chuẩn truyền thông SPI. Chuẩn SPI cho phép truyền dữ liệu với tốc độ cao và phù hợp với các ứng dụng điều khiển LED ma trận.

Các chân kết nối chính bao gồm:

- + Chân DIN của MAX7219 kết nối với chân MOSI của ESP8266 để truyền dữ liệu hiển thị.
- + Chân CLK kết nối với chân Clock để đồng bộ xung truyền dữ liệu.
- + Chân CS kết nối với chân điều khiển Chip Select để lựa chọn module LED.
- + Chân VCC và GND dùng để cấp nguồn cho module LED Matrix.

Trong hệ thống, ESP8266 nhận dữ liệu văn bản từ ứng dụng Flutter thông qua WiFi, sau đó xử lý và gửi dữ liệu hiển thị đến LED Matrix qua giao tiếp SPI.

### 3.2.2. Sơ đồ kết nối phần cứng



Hình 2. Sơ đồ kết nối phần cứng

Sơ đồ kết nối phần cứng mô tả cách liên kết giữa ESP8266 NodeMCU và module LED Matrix MAX7219. Hệ thống sử dụng nguồn cấp 3V cho LED Matrix và nguồn cho ESP8266.

Các kết nối chính trong hệ thống gồm:

- + NodeMCU D8 → CS của led
- + NodeMCU D7 → DIN của led
- + NodeMCU D5 → CLK của led
- + GND → GND
- + 3V → VCC

Thông qua sơ đồ kết nối này, ESP8266 có thể gửi dữ liệu điều khiển trực tiếp đến LED Matrix để thực hiện các hiệu ứng hiển thị như chạy chữ, nhấp nháy, hiện từng ký tự và hiển thị thời gian thực.

### 3.3. Thiết kế phần mềm

Phần mềm được thiết kế gồm hai thành phần chính: chương trình điều khiển chạy trên ESP8266 NodeMCU và ứng dụng điều khiển được xây dựng bằng Flutter trên thiết bị di động. Hai thành phần này giao tiếp với nhau thông qua mạng WiFi bằng giao thức HTTP. Ứng dụng Flutter đóng vai trò gửi lệnh điều khiển, trong khi ESP8266 tiếp nhận, xử lý dữ liệu và điều khiển LED Matrix MAX7219 hiển thị nội dung tương ứng.

Phần mềm của hệ thống được thiết kế theo hướng đơn giản, dễ triển khai và phù

hợp với các ứng dụng IoT quy mô nhỏ. Các tham số điều khiển như nội dung văn bản, tốc độ chạy chữ, độ sáng, số vòng lặp, hiệu ứng, trạng thái nhấp nháy và lệnh dừng được truyền từ ứng dụng Flutter đến ESP8266 thông qua URL. Sau khi nhận dữ liệu, ESP8266 phân tích các tham số và thực hiện cập nhật trạng thái hiển thị trên LED Matrix.

### 3.3.1. Thiết kế chương trình ESP8266

Chương trình ESP8266 được xây dựng bằng Arduino IDE, sử dụng các thư viện chính gồm ESP8266WiFi, WiFiClient và LEDMatrixDriver. ESP8266 đóng vai trò là bộ điều khiển trung tâm của hệ thống, vừa đảm nhiệm kết nối mạng WiFi, vừa xử lý request từ ứng dụng Flutter và điều khiển LED Matrix.

Khi khởi động, ESP8266 thực hiện kết nối đến mạng WiFi đã được cấu hình sẵn bằng SSID và mật khẩu. Sau khi kết nối thành công, ESP8266 khởi tạo Web Server trên cổng 80 và hiển thị địa chỉ IP của thiết bị. Địa chỉ IP này được sử dụng để ứng dụng Flutter gửi các lệnh điều khiển đến ESP8266.

Chương trình ESP8266 được thiết kế gồm các nhóm chức năng chính như sau:

- + Khởi tạo WiFi và Web Server: ESP8266 kết nối vào mạng WiFi và lắng nghe các yêu cầu HTTP từ ứng dụng di động.
- + Xử lý HTTP Request: Khi nhận request từ Flutter, ESP8266 tách các tham số truyền qua URL như text, speed, brightness, loops, blink, effect và stop.
- + Xử lý văn bản hiển thị: Dữ liệu văn bản được giải mã URL, loại bỏ ký tự không phù hợp và lưu vào biến hiển thị. Hệ thống sử dụng bộ font 8x8 để ánh xạ từng ký tự thành dạng điểm ảnh trên LED Matrix.
- + Điều khiển LED Matrix: ESP8266 sử dụng thư viện LEDMatrixDriver để gửi dữ liệu điều khiển đến LED Matrix MAX7219 thông qua giao tiếp SPI.
- + Điều chỉnh tham số hiển thị: Người dùng có thể thay đổi tốc độ chạy chữ, độ sáng LED, số vòng lặp và trạng thái nhấp nháy thông qua ứng dụng Flutter.
- + Xử lý hiệu ứng hiển thị: Hệ thống hỗ trợ hiệu ứng chữ chạy ngang và hiệu ứng hiện từng chữ. Đối với hiệu ứng hiện từng chữ, khi nội dung vượt quá chiều rộng LED Matrix, hệ thống chỉ hiển thị một cửa sổ gồm tối đa bốn ký tự và tự động dịch sang các ký tự tiếp theo.
- + Lệnh dừng hiển thị: Khi nhận tham số stop, ESP8266 chuyển trạng thái hệ thống sang STOPPED, xóa nội dung trên LED Matrix và tạm dừng hiển thị.

Vòng lặp chính của chương trình liên tục thực hiện hai nhiệm vụ: cập nhật trạng thái hiển thị LED và kiểm tra request mới từ client. Nhờ đó, hệ thống có thể vừa duy trì hiệu ứng hiển thị hiện tại, vừa phản hồi các lệnh điều khiển mới từ ứng dụng Flutter.

### 3.3.2. Thiết kế ứng dụng Flutter

Ứng dụng Flutter được thiết kế theo dạng dashboard điều khiển tập trung, giúp người dùng dễ dàng thao tác và theo dõi trạng thái điều khiển của hệ thống. Giao diện ứng dụng được xây dựng bằng Material Design với nền tối, các nhóm chức năng được bố trí theo dạng thẻ, giúp tăng tính trực quan và thuận tiện khi sử dụng.

Các thành phần chính của ứng dụng Flutter bao gồm:

- + Trường nhập địa chỉ IP của ESP8266: Cho phép người dùng nhập địa chỉ IP của mạch LED để thiết lập kết nối.
- + Trường nhập văn bản hiển thị: Cho phép nhập nội dung cần hiển thị trên LED Matrix. Trước khi gửi, văn bản được xử lý để loại bỏ dấu tiếng Việt và các ký tự không thuộc bảng ASCII nhằm tránh lỗi hiển thị.
- + Thanh điều chỉnh tốc độ chạy chữ: Cho phép thay đổi tốc độ hiệu ứng hiển thị. Giá trị càng nhỏ thì chữ chạy càng nhanh.
- + Thanh điều chỉnh độ sáng: Cho phép thay đổi độ sáng LED Matrix trong khoảng từ 0 đến 15.
- + Thanh điều chỉnh số vòng lặp: Cho phép thiết lập số lần lặp lại nội dung hiển thị.
- + Công tắc hiệu ứng hiện từng chữ: Khi bật, ứng dụng gửi tham số effect đến ESP8266 để kích hoạt chế độ hiện từng chữ.
- + Công tắc nhấp nháy: Cho phép bật hoặc tắt hiệu ứng Blink trên LED Matrix.
- + Công tắc dừng hiển thị: Cho phép gửi lệnh dừng hoạt động hiển thị đến ESP8266.
- + Nút hiển thị giờ hiện tại: Ứng dụng lấy thời gian hiện tại từ thiết bị di động và gửi xuống ESP8266 để hiển thị lên LED Matrix.
- + Chức năng hẹn giờ tự động: Người dùng có thể chọn thời điểm thực thi. Ứng dụng lưu lại nội dung và các thông số tại thời điểm đặt lịch, sau đó tự động gửi lệnh đến ESP8266 khi đến giờ.
- + Chế độ khẩn cấp: Khi kích hoạt Emergency Mode, ứng dụng tự động gửi nội dung SOS SOS, bật nhấp nháy, đặt độ sáng tối đa, tăng số vòng lặp và khóa các thành phần chỉnh sửa để tránh thao tác nhầm. Khi nhấn dừng khẩn cấp, ứng dụng gửi lệnh dừng hiển thị và mở khóa giao diện.

Thiết kế phần mềm Flutter giúp hệ thống có giao diện điều khiển trực quan, hỗ trợ nhiều chức năng điều khiển LED và có khả năng mở rộng thêm các tính năng IoT khác trong tương lai.

### 3.4. Thiết kế giao tiếp giữa Flutter App và ESP8266

Hệ thống sử dụng giao thức HTTP để giao tiếp giữa ứng dụng Flutter và ESP8266 thông qua mạng WiFi nội bộ. Ứng dụng Flutter đóng vai trò gửi các yêu

cầu điều khiển đến địa chỉ IP của ESP8266 dưới dạng HTTP GET request. Sau khi nhận request, ESP8266 tiến hành phân tích các tham số trong URL và thực hiện điều khiển LED Matrix tương ứng.

Mô hình giao tiếp của hệ thống có ưu điểm đơn giản, dễ triển khai và phù hợp với các ứng dụng IoT quy mô nhỏ. Toàn bộ dữ liệu điều khiển được truyền trực tiếp qua URI dưới dạng các tham số như nội dung hiển thị, tốc độ chạy chữ, độ sáng, số vòng lặp, hiệu ứng nhấp nháy và lệnh dừng hiển thị.

#### 3.4.1. Luồng gửi dữ liệu

Luồng gửi dữ liệu của hệ thống bắt đầu từ ứng dụng Flutter. Khi người dùng nhập nội dung và nhấn nút điều khiển, ứng dụng sẽ lấy dữ liệu từ giao diện, xử lý dữ liệu đầu vào và gửi request đến ESP8266.

Trong ứng dụng Flutter, quá trình gửi dữ liệu được thực hiện trong hàm `_executeRequest()` như sau:

```
try {  
  final String encodedText = Uri.encodeComponent(text);  
  
  final int effectValue = _enableTypeEffect ? 1 : 0;  
  
  String urlString =  
  
  'http://$ip/?text=$encodedText&speed=${speed.toInt()}&brightness=${brightness.toInt()}&loops=${loops.toInt()}&effect=$effectValue';  
  if (enableBlink) urlString += '&blink=1';  
  if (stopDisplay) urlString += '&stop=1';  
  urlString += '&end=1';  
  
  final Uri url = Uri.parse(urlString);  
  final response = await http.get(url).timeout(const Duration(seconds: 5));
```

Giải thích:

- + Mã hóa nội dung bằng `Uri.encodeComponent()` tránh lỗi ký tự đặc biệt.
- + Tạo URI chứa các tham số điều khiển.
- + Gửi HTTP GET request đến ESP8266 thông qua thư viện `http`.

Sau khi request được gửi đi, ESP8266 sẽ nhận dữ liệu từ Web Server bằng đối tượng `WiFiClient`.

```
#Code Arduino  
WiFiClient client = server.available();  
if (!client) return;  
...  
String req = client.readStringUntil('\r');  
...  
handleRequest(req);
```

Giải thích:

- + Kiểm tra xem có client kết nối đến Web Server hay không.
- + Đọc nội dung request HTTP từ Flutter.
- + Chuyển request sang hàm `handleRequest()` để xử lý dữ liệu điều khiển.

Sau khi phân tích request, ESP8266 cập nhật trạng thái hiển thị trên LED Matrix như tốc độ chạy chữ, độ sáng, hiệu ứng hoặc nội dung hiển thị mới.

### 3.4.2. Thiết kế URI điều khiển

Hệ thống sử dụng URI để truyền dữ liệu điều khiển từ Flutter đến ESP8266. Các tham số được gắn trực tiếp vào URL theo dạng `key=value`.

URI điều khiển:

```
http://<IP_ESP8266>/?text=<noi_dung>&speed=<toc_do>&brightness=<do_sang>&loops=<so_vong_lap>&effect=<hie_u_ung>&blink=1
```

Trong đó:

- + `text`: nội dung hiển thị.
- + `speed`: tốc độ chạy chữ.
- + `brightness`: độ sáng LED Matrix.
- + `loops`: số vòng lặp hiển thị.
- + `effect`: hiệu ứng hiển thị.
- + `blink`: bật/tắt nhấp nháy.

Phía Flutter thực hiện tạo URI bằng cách ghép chuỗi tham số:

```
#Code Flutter
String urlString =

'http://$ip/?text=$encodedText&speed=${speed.toInt()}&brightness=${brightness.toInt()}&loops=${loops.toInt()}&effect=$effectValue';
if (enableBlink) urlString += '&blink=1';
if (stopDisplay) urlString += '&stop=1';
urlString += '&end=1';
```

Đoạn code trên cho phép hệ thống linh hoạt bổ sung hoặc loại bỏ các tham số điều khiển tùy theo trạng thái giao diện người dùng.

Ở phía ESP8266, hệ thống sử dụng hàm `getParam()` để tách giá trị của từng tham số trong URI.

```
#Code Arduino
String getParam(String params, String key) {
    String searchKey = key + "=";
    int start = params.indexOf(searchKey);
    if (start < 0) return "";
```

```

start += searchKey.length();

int end = params.indexOf('&', start);
if (end < 0) end = params.indexOf(' ', start);
if (end < 0) end = params.length();

return params.substring(start, end);
}

```

Hàm trên thực hiện tìm kiếm tên tham số trong URL và trả về giá trị tương ứng.

Sau khi lấy được dữ liệu, ESP8266 tiến hành xử lý từng tham số:

```

#Code Arduino
String speedStr = getParam(params, "speed");
if (speedStr.length() > 0) {
    animationSpeed = speedStr.toInt();
    if (animationSpeed < 10) animationSpeed = 10;
    if (animationSpeed > 500) animationSpeed = 500;
}

String brightnessStr = getParam(params, "brightness");
if (brightnessStr.length() > 0) {
    brightness = brightnessStr.toInt();
    if (brightness < 0) brightness = 0;
    if (brightness > 15) brightness = 15;
    lmd.setIntensity(brightness);
}

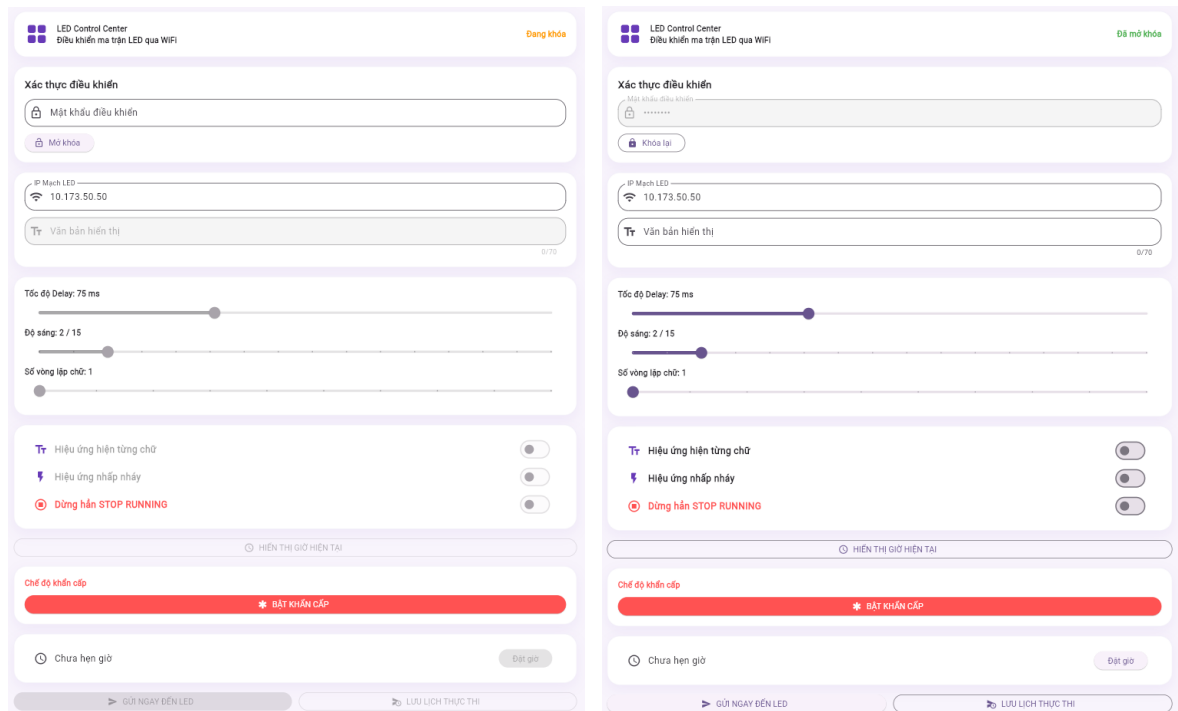
```

Đoạn code trên cho phép ESP8266 thay đổi trực tiếp tốc độ chạy chữ và độ sáng LED Matrix theo dữ liệu nhận được từ ứng dụng Flutter. Thiết kế giao tiếp bằng URI giúp hệ thống dễ mở rộng thêm các chức năng mới trong tương lai mà không cần thay đổi cấu trúc giao tiếp tổng thể.

### 3.5. Thiết kế giao diện người dùng

Giao diện người dùng của ứng dụng được thiết kế bằng Flutter theo dạng Dashboard điều khiển tập trung. Mục tiêu của giao diện là giúp người dùng có thể nhập nội dung, điều chỉnh các thông số hiển thị và gửi lệnh điều khiển đến LED Matrix một cách trực quan, nhanh chóng.

Ứng dụng sử dụng giao diện nền tối, các thành phần được chia thành từng nhóm chức năng riêng biệt như nhập dữ liệu, điều chỉnh tham số, chọn hiệu ứng, hẹn giờ và chế độ khẩn cấp. Cách bố trí này giúp người dùng dễ quan sát và hạn chế thao tác nhầm trong quá trình sử dụng.



Hình 3. Giao diện người dùng khi đang khoá và mở khoá

### 3.5.1. Các thành phần giao diện chính

Giao diện ứng dụng bao gồm nhiều thành phần điều khiển khác nhau, mỗi thành phần đảm nhiệm một chức năng riêng trong quá trình điều khiển LED Matrix. Các thành phần giao diện được xây dựng nhằm hỗ trợ người dùng thao tác trực tiếp với hệ thống LED Matrix thông qua flutter. Mỗi thành phần đảm nhiệm một chức năng điều khiển riêng biệt như nhập dữ liệu, thiết lập hiệu ứng hiển thị, điều chỉnh thông số và gửi lệnh điều khiển.

#### a) Ô nhập địa chỉ IP của ESP8266

Ô nhập IP cho phép người dùng nhập địa chỉ IP của ESP8266 trong mạng WiFi nội bộ. Địa chỉ này được sử dụng để ứng dụng gửi HTTP request đến đúng thiết bị.

```
_inputField(
  controller: _ipController,
  label: 'IP Mạch LED',
  icon: Icons.wifi,
),
```

#### b) Ô nhập văn bản hiển thị

Ô nhập văn bản cho phép người dùng nhập nội dung cần hiển thị trên LED Matrix. Nội dung này sẽ được xử lý và gửi xuống ESP8266 khi người dùng nhấn nút gửi.

```
_inputField(
  controller: _textController,
  label: 'Văn bản hiển thị',
  icon: Icons.text_fields,
  enabled: !_isEmergencyMode,
```

```
maxLength: 70,  
),
```

Trong đoạn code trên, thuộc tính `enabled` được gán với biến `_isEmergencyMode`. Khi chế độ khẩn cấp được kích hoạt, ô nhập văn bản sẽ bị khóa để tránh người dùng thay đổi nội dung ngoài ý muốn.

### c) Thanh điều chỉnh tốc độ, độ sáng và số vòng lặp

Ứng dụng sử dụng Slider để người dùng thay đổi các thông số hiển thị như tốc độ chạy chữ, độ sáng LED và số vòng lặp. Cơ chế tương tự được áp dụng cho thanh điều chỉnh độ sáng (`_brightness`) và số vòng lặp (`_loops`).

```
_sliderItem(  
  title: 'Tốc độ Delay',  
  valueText: '${_speed.toInt()} ms',  
  value: _speed,  
  min: 10,  
  max: 200,  
  divisions: 190,  
  onChanged: _isEmergencyMode  
    ? null  
    : (val) => setState(() => _speed = val),  
),
```

Cơ chế tương tự cũng được áp dụng cho thanh điều chỉnh độ sáng và số vòng lặp. Khi Emergency Mode đang bật, các Slider bị vô hiệu hóa nhằm khóa chỉnh sửa.

### d) Công tắc hiệu ứng hiển thị

Các hiệu ứng như hiện từng chữ, nhấp nháy và dừng hiển thị được điều khiển bằng SwitchListTile. Người dùng có thể bật hoặc tắt từng chức năng một cách trực quan.

```
_switchItem(  
  icon: Icons.flash_on_rounded,  
  title: 'Hiệu ứng nhấp nháy',  
  value: _enableBlink,  
  onChanged: _isEmergencyMode  
    ? null  
    : (val) => setState(() => _enableBlink = val),  
),
```

Đoạn code trên cho thấy công tắc Blink sẽ được khóa khi hệ thống đang ở chế độ khẩn cấp. Điều này giúp đảm bảo trạng thái cảnh báo không bị thay đổi trong quá trình sử dụng.

### e) Nút hiển thị giờ hiện tại

Nút hiển thị giờ hiện tại cho phép ứng dụng lấy thời gian từ thiết bị di động và gửi xuống ESP8266 để hiển thị trên LED Matrix.

```
child: OutlinedButton.icon(
  onPressed:
    (_isLoading || _isEmergencyMode) ? null : _sendCurrentTime,
  icon: const Icon(Icons.access_time),
  label: const Text(
    'HIỂN THỊ GIỜ HIỆN TẠI',
    style: TextStyle(fontWeight: FontWeight.bold),
  ),
),
```

Khi người dùng nhấn nút này, ứng dụng gọi hàm `_sendCurrentTime()` để tạo chuỗi thời gian theo định dạng giờ:phút và gửi xuống LED Matrix.

#### f) Khu vực chế độ khẩn cấp

Chế độ khẩn cấp được thiết kế thành một khối giao diện riêng với màu đỏ nhằm tạo sự nổi bật. Khi người dùng nhấn nút bật khẩn cấp, hệ thống sẽ gửi nội dung cảnh báo SOS SOS, bật nhấp nháy, tăng độ sáng tối đa và khóa các thành phần chỉnh sửa.

```
child: ElevatedButton.icon(
  onPressed: _isLoading ? null : _sendEmergency,
  icon: const Icon(Icons.emergency),
  label: const Text(
    'BẬT KHẨN CẤP',
    style: TextStyle(fontWeight: FontWeight.bold),
  ),
),
```

Khi chế độ khẩn cấp đang bật, giao diện chuyển sang nút dừng khẩn cấp để người dùng có thể tắt trạng thái cảnh báo.

```
child: OutlinedButton.icon(
  onPressed: _isLoading ? null : _stopEmergency,
  icon: const Icon(Icons.stop_circle_outlined),
  label: const Text(
    'DỪNG KHẨN CẤP',
    style: TextStyle(fontWeight: FontWeight.bold),
  ),
),
```

#### g) Khu vực hẹn giờ

Khu vực hẹn giờ cho phép người dùng chọn thời điểm gửi nội dung xuống LED Matrix. Khi đến thời gian đã chọn, ứng dụng tự động gửi nội dung và thông số đã lưu đến ESP8266.

```
ElevatedButton.icon(
  onPressed: _isEmergencyMode
    ? null
    : () => _selectTime(context),
  icon: const Icon(Icons.access_time, size: 18),
),
```

```
label: const Text('Đặt giờ'),  
)
```

#### h) Nút gửi dữ liệu và lưu lịch

Hai nút chức năng chính của ứng dụng là gửi dữ liệu ngay đến LED Matrix và lưu lịch thực thi. Các nút này được đặt ở cuối giao diện để người dùng dễ thao tác sau khi đã thiết lập đầy đủ thông số.

```
child: ElevatedButton.icon(  
  onPressed:  
    (_isLoading || _isEmergencyMode) ? null : _sendNow,  
  icon: _isLoading  
    ? const SizedBox(  
      width: 22,  
      height: 22,  
      child:  
        CircularProgressIndicator(strokeWidth: 2),  
    )  
    : const Icon(Icons.send_rounded),  
  label: const Text(  
    'GỬI NGAY ĐẾN LED',  
    style: TextStyle(fontWeight: FontWeight.bold),  
  ),  
)
```

Nhìn chung, giao diện được thiết kế theo hướng đơn giản, trực quan và dễ sử dụng. Các thành phần điều khiển được nhóm rõ ràng, hỗ trợ đầy đủ các chức năng của hệ thống như gửi văn bản, điều chỉnh hiệu ứng, hẹn giờ, hiển thị thời gian và chế độ khẩn cấp.

## CHƯƠNG 4. HIỆN THỰC HỆ THỐNG

### 4.1. Cài đặt môi trường phát triển

Trước khi tiến hành xây dựng hệ thống, nhóm thực hiện cài đặt các công cụ và môi trường cần thiết nhằm phục vụ cho việc lập trình phần cứng ESP8266 và phát triển ứng dụng điều khiển trên nền tảng Flutter. Các công cụ được sử dụng bao gồm Arduino IDE, Flutter SDK và Android Studio.

#### 4.1.1. Cài đặt Arduino IDE

Arduino IDE được sử dụng để lập trình và nạp chương trình cho vi điều khiển ESP8266 NodeMCU. Phần mềm hỗ trợ biên dịch mã nguồn, quản lý thư viện và giao tiếp với thiết bị thông qua cổng COM.

Quá trình cài đặt bao gồm các bước:

- + Tải và cài đặt Arduino IDE từ trang chủ Arduino.
- + Thêm Board Manager URL của ESP8266 vào phần Preferences.
- + Cài đặt thư viện ESP8266 Board thông qua Board Manager.
- + Cài đặt các thư viện hỗ trợ như LEDMatrixDriver và ESP8266WiFi.
- + Kết nối ESP8266 với máy tính và lựa chọn đúng loại board, cổng COM trước khi nạp chương trình.

Sau khi hoàn tất, Arduino IDE được sử dụng để xây dựng Web Server trên ESP8266, xử lý HTTP Request và điều khiển LED Matrix.

#### 4.1.2. Cài đặt Flutter SDK và Android Studio

Flutter SDK và Android Studio được sử dụng để phát triển ứng dụng điều khiển LED Matrix trên thiết bị di động.

Quá trình cài đặt bao gồm:

- + Tải và cài đặt Flutter SDK.
- + Thiết lập biến môi trường PATH cho Flutter.
- + Cài đặt Android Studio và Android SDK.
- + Cài đặt plugin Flutter và Dart trong Android Studio.
- + Kiểm tra môi trường bằng lệnh flutter doctor để đảm bảo các thành phần đã được cấu hình đầy đủ.

Sau khi hoàn tất cài đặt, Android Studio được sử dụng để thiết kế giao diện người dùng, xây dựng chức năng điều khiển LED qua WiFi và biên dịch ứng dụng chạy trên thiết bị Android.

### 4.2. Lập trình ESP8266

ESP8266 NodeMCU đóng vai trò là bộ điều khiển trung tâm của hệ thống. Chương trình trên ESP8266 được xây dựng bằng Arduino IDE, sử dụng các thư viện hỗ trợ kết nối WiFi, tạo Web Server và điều khiển LED Matrix MAX7219.

Chương trình ESP8266 thực hiện các nhiệm vụ chính gồm: kết nối vào mạng WiFi, khởi tạo Web Server, nhận và xử lý HTTP Request từ ứng dụng Flutter, sau đó điều khiển LED Matrix hiển thị nội dung theo các tham số nhận được.

#### 4.2.1. Kết nối WiFi

Đầu tiên, chương trình khai báo tên mạng WiFi và mật khẩu để ESP8266 có thể kết nối vào mạng nội bộ. Sau khi kết nối thành công, ESP8266 sẽ lấy địa chỉ IP để ứng dụng Flutter gửi lệnh điều khiển đến thiết bị.

```
#Code Arduino
// ===== CẤU HÌNH WIFI =====
const char* ssid = "Tuong Van";
const char* password = "30042005";

WiFi.begin(ssid, password);

while (WiFi.status() != WL_CONNECTED) {
    displayText(text);
    delay(ANIM_DELAY);
}
```

Đoạn code trên thực hiện kết nối ESP8266 đến mạng WiFi đã cấu hình. Trong khi chờ kết nối, hệ thống vẫn gọi hàm `displayText()` để hiển thị trạng thái kết nối trên LED Matrix.

Sau khi kết nối thành công, chương trình lấy địa chỉ IP của ESP8266:

```
ip = WiFi.localIP();
Serial.println(ip);
```

Địa chỉ IP này được dùng làm địa chỉ truy cập từ ứng dụng Flutter hoặc trình duyệt web.

#### 4.2.2. Xây dựng Web Server

ESP8266 được thiết kế hoạt động như một Web Server đơn giản trên cổng 80. Web Server này có nhiệm vụ lắng nghe các request từ ứng dụng Flutter hoặc trình duyệt web.

```
WiFiServer server(80);
```

Trong hàm `setup()`, Web Server được khởi động sau khi ESP8266 kết nối WiFi thành công:

```
server.begin();
```

Trong vòng lặp chính, ESP8266 kiểm tra xem có client nào gửi request đến hay không:

```
WiFiClient client = server.available();
if (!client) return;
```

Nếu không có client kết nối, chương trình tiếp tục duy trì việc hiển thị LED Matrix. Khi có client gửi request đến, ESP8266 sẽ đọc dữ liệu request để xử lý các lệnh điều khiển.

#### 4.2.3. Xử lý HTTP Request

Khi nhận được request từ client, ESP8266 đọc dòng đầu tiên của HTTP Request và chuyển nội dung này sang hàm `handleRequest()` để xử lý.

```
String req = client.readStringUntil('\r');
client.flush();

Serial.println(req);
```

Trong hàm `handleRequest()`, chương trình kiểm tra request có chứa tham số hay không. Nếu có, phần tham số sau dấu `?` sẽ được tách ra để xử lý.

```
if (req.indexOf('?') <= 0) return;

String params = req.substring(req.indexOf('?') + 1);
```

Để lấy giá trị từng tham số trong URL, chương trình sử dụng hàm `getParam()`:

```
String getParam(String params, String key) {
    String searchKey = key + "=";
    int start = params.indexOf(searchKey);
    if (start < 0) return "";

    start += searchKey.length();

    int end = params.indexOf('&', start);
    if (end < 0) end = params.indexOf(' ', start);
    if (end < 0) end = params.length();

    return params.substring(start, end);
}
```

Hàm này tìm kiếm tên tham số trong URL và trả về giá trị tương ứng. Ví dụ, chương trình có thể lấy các tham số như `text`, `speed`, `brightness`, `loops`, `blink`, `stop` và `effect`.

Sau khi lấy được tham số `text`, ESP8266 giải mã nội dung và lưu vào biến hiển thị:

```
String newText = getParam(params, "text");
if (newText.length() > 0) {
    newText = urlDecode(newText);
    safeCopyText(newText);

    x = LEDMATRIX_WIDTH;
    currentLoop = 0;
    displayOn = true;
    currentState = RUNNING;
    lmd.clear();
}
```

Đoạn code trên giúp ESP8266 nhận nội dung mới từ Flutter, giải mã chuỗi URL và đặt lại trạng thái hiển thị để LED Matrix bắt đầu chạy nội dung mới.

#### 4.2.4. Điều khiển LED Matrix

LED Matrix được điều khiển thông qua thư viện LEDMatrixDriver. Trong chương trình, hệ thống sử dụng 4 module LED 8x8 ghép lại thành ma trận 32x8.

```
// ===== CẤU HÌNH LED MAX7219 =====  
const int LEDMATRIX_SEGMENTS = 4;  
const int LEDMATRIX_WIDTH = 32;  
const int LEDMATRIX_HEIGHT = 8;  
  
LEDMatrixDriver lmd(LEDMATRIX_SEGMENTS, LEDMATRIX_CS_PIN);
```

Trong hàm setup(), LED Matrix được kích hoạt và thiết lập độ sáng ban đầu:

```
lmd.setEnabled(true);  
lmd.setIntensity(brightness);
```

Để hiển thị ký tự lên LED Matrix, chương trình sử dụng bộ font 8x8. Mỗi ký tự được ánh xạ thành một mảng byte, sau đó hàm drawSprite() sẽ vẽ từng điểm ảnh lên ma trận LED.

```
void drawSprite(byte* sprite, int x, int y, int width, int height) {  
    for (int iy = 0; iy < height; iy++) {  
        byte mask = B10000000;  
  
        for (int ix = 0; ix < width; ix++) {  
            bool pixel = sprite[iy] & mask;  
  
            int px = x + (width - 1 - ix);  
            int py = y + iy;  
  
            if (px >= 0 && px < LEDMATRIX_WIDTH && py >= 0 && py <  
LEDMATRIX_HEIGHT) {  
                lmd.setPixel(px, py, pixel);  
            }  
  
            mask >>= 1;  
        }  
    }  
}
```

Hàm drawString() có nhiệm vụ duyệt qua từng ký tự trong chuỗi, tìm vị trí font tương ứng và gọi drawSprite() để hiển thị ký tự đó.

```
void drawString(char* textPtr, int lenText, int xPos, int yPos) {  
    for (int idx = 0; idx < lenText; idx++) {  
        char ch = textPtr[idx];  
  
        if (ch < 32 || ch > 126) continue;
```

```

int c = ch - 32;

if (c < 0 || c >= 95) continue;

int charX = xPos + idx * 8;

if (charX >= LEDMATRIX_WIDTH) continue;
if (charX + 7 < 0) continue;

drawSprite(font[c], charX, yPos, 8, 8);
}
}

```

Việc hiển thị văn bản được thực hiện trong hàm `displayText()`. Ở chế độ mặc định, chữ được hiển thị theo dạng chạy ngang từ phải sang trái.

```

if (textEffect == 0) {
    if (millis() - myTime < animationSpeed) return;
    myTime = millis();

    lmd.clear();

    if (showText) {
        drawString(theText, len, x, 0);
    }

    lmd.display();
    x--;
    ....
}

```

Bên cạnh đó, chương trình còn hỗ trợ hiệu ứng hiện từng chữ theo dạng cửa sổ bốn ký tự. Khi nội dung dài hơn chiều rộng LED Matrix, hệ thống tự động chuyển sang nhóm ký tự tiếp theo để tránh mất nội dung.

```

int maxChars = LEDMATRIX_WIDTH / 8; // 32 / 8 = 4 ký tự
int startIndex = 0;

if (typeIndex > maxChars) {
    startIndex = typeIndex - maxChars;
}

```

Ngoài việc hiển thị văn bản, ESP8266 còn xử lý các chức năng điều khiển như điều chỉnh độ sáng, nhấp nháy và dừng hiển thị. Ví dụ, khi nhận lệnh dừng từ Flutter, chương trình chuyển trạng thái sang STOPPED và xóa màn hình LED:

```

if (stopStr == "1") {
    currentState = STOPPED;
    displayOn = false;
}

```

```
lmd.clear();  
lmd.display();  
}
```

Như vậy, ESP8266 đảm nhận toàn bộ quá trình nhận lệnh, xử lý tham số và điều khiển LED Matrix. Đây là thành phần trung tâm giúp hệ thống bảng tin điện tử có thể hoạt động linh hoạt theo các thao tác từ ứng dụng di động.

### 4.3. Hiệu ứng và chức năng hiển thị trên LED Matrix

Hệ thống hỗ trợ nhiều chức năng hiển thị khác nhau nhằm tăng tính trực quan và linh hoạt cho bảng LED Matrix. Các chức năng này được xây dựng kết hợp giữa ứng dụng Flutter và chương trình xử lý trên ESP8266.

#### 4.3.1. Xử lý văn bản hiển thị

Ứng dụng Flutter cho phép người dùng nhập nội dung cần hiển thị, sau đó gửi xuống ESP8266 thông qua HTTP Request.

```
#Code Flutter  
Future<void> _sendNow() async  
{  
  final String ip = _ipController.text.trim();  
  final String rawText = _textController.text.trim();  
  
  ....  
  
  await _executeRequest(  
    ip: ip,  
    text: finalCleanText,  
    speed: _speed,  
    brightness: _brightness,  
    loops: _loops,  
    enableBlink: _enableBlink,  
    stopDisplay: _stopDisplay,  
    clearInputAfterSend: true,  
  );  
}
```

Trước khi gửi dữ liệu, ứng dụng gọi hàm `_sanitizeText()` để loại bỏ dấu tiếng Việt và các ký tự không hỗ trợ.

Ở phía ESP8266, nội dung nhận được sẽ được giải mã và lưu vào biến hiển thị:

```
void handleRequest(String req) {  
  ...  
  String newText = getParam(params, "text");  
  if (newText.length() > 0) {  
    newText = urlDecode(newText);  
    safeCopyText(newText);  
    .... }  
}
```

Sau đó nội dung được hiển thị lên LED Matrix thông qua hàm `drawString()`.

### 4.3.2. Điều chỉnh tốc độ chạy chữ

Người dùng có thể thay đổi tốc độ chạy chữ bằng Slider trên giao diện Flutter.

```
#Code Flutter
Slider(
  value: _speed,
  min: 10,
  max: 200,
  divisions: 190,
  label: '${_speed.toInt()} ms',
  // onChanged: (val) => setState(() => _speed = val),
  onChanged: _isEmergencyMode ? null : (val) => setState(() => _speed = val),
),
```

Giá trị tốc độ được gửi xuống ESP8266 qua tham số speed.

```
#Code Arduino
String speedStr = getParam(params, "speed");
if (speedStr.length() > 0) {
  animationSpeed = speedStr.toInt();
  if (animationSpeed < 10) animationSpeed = 10;
  if (animationSpeed > 500) animationSpeed = 500;
}
```

Trong quá trình hiển thị, biến animationSpeed được sử dụng để điều chỉnh thời gian cập nhật hiệu ứng chạy chữ.

```
if (textEffect == 0) {
  if (millis() - myTime < animationSpeed) return;
  myTime = millis();
  ...
}
```

Giá trị càng nhỏ thì chữ chạy càng nhanh.

### 4.3.3. Điều chỉnh độ sáng LED Matrix

Ứng dụng Flutter hỗ trợ thay đổi độ sáng LED Matrix từ mức 0 đến 15.

```
#Code Flutter
Slider(
  value: _brightness,
  min: 0,
  max: 15,
  divisions: 15,
  label: '${_brightness.toInt()}',
  onChanged: _isEmergencyMode ? null : (val) => setState(() => _brightness = val),
),
```

ESP8266 nhận giá trị độ sáng và thiết lập cường độ sáng cho MAX7219.

```
#Code Arduino
String brightnessStr = getParam(params, "brightness");
if (brightnessStr.length() > 0) {
    brightness = brightnessStr.toInt();
    if (brightness < 0) brightness = 0;
    if (brightness > 15) brightness = 15;
    lmd.setIntensity(brightness);
}
```

Hàm `setIntensity()` của thư viện `LEDMatrixDriver` giúp điều chỉnh độ sáng LED theo thời gian thực.

#### 4.3.4. Thiết lập số vòng lặp chữ

Hệ thống hỗ trợ thiết lập số lần lặp lại nội dung hiển thị trên LED Matrix.

```
#Code Flutter
Slider(
  value: _loops,
  min: 1,
  max: 10,
  divisions: 9,
  label: '${_loops.toInt()}',
  onChanged: _isEmergencyMode ? null : (val) => setState(() => _loops = val),
),
```

ESP8266 xử lý tham số `loops` như sau:

```
String loopsStr = getParam(params, "loops");
if (loopsStr.length() > 0) {
    numLoops = loopsStr.toInt();
    if (numLoops < 1) numLoops = 1;
    currentLoop = 0;
}
```

Sau mỗi lần chạy hết nội dung, biến `currentLoop` sẽ tăng lên để kiểm tra đã đủ số vòng lặp hay chưa.

#### 4.3.5. Hiệu ứng hiện từng chữ

Bên cạnh hiệu ứng chữ chạy ngang mặc định, hệ thống còn hỗ trợ hiệu ứng hiện từng chữ. Khi người dùng bật hiệu ứng này trên ứng dụng Flutter, app sẽ gửi tham số `effect` xuống ESP8266 để kích hoạt chế độ hiển thị từng ký tự.

Đoạn mã xử lý phía Flutter:

```
final int effectValue = _enableTypeEffect ? 1 : 0;

String urlString =
'http://$ip/?text=$encodedText&speed=${speed.toInt()}&brightness=${brightness.toInt()}&loops=${loops.toInt()}&effect=$effectValue';
```

Ở phía ESP8266, khi `textEffect` có giá trị bằng 1, chương trình sẽ tăng dần số ký

tự được hiển thị thông qua biến typeIndex.

Đoạn mã xử lý phía ESP8266:

```
if (textEffect == 1)
{
    unsigned long now = millis();

    if (now - effectTime >= TYPE_DELAY) {
        effectTime = now;

        if (typeIndex < len) {
            typeIndex++;
        } else
        {
            delay(800);

            currentLoop++;

            if (currentLoop < numLoops) {
                typeIndex = 0;
            } else {
                currentLoop = 0;
                currentState = STOPPED;
                lmd.clear();
                lmd.display();
                return;
            }
        }
    }
}
```

Do LED Matrix có kích thước 32x8, mỗi ký tự dùng font 8x8 nên màn hình chỉ hiển thị được tối đa 4 ký tự cùng lúc. Vì vậy, chương trình sử dụng cơ chế cửa sổ hiển thị để xử lý văn bản dài vượt khung.

Đoạn mã xử lý phía ESP8266:

```
int maxChars = LEDMATRIX_WIDTH / 8; // 32 / 8 = 4 ký tự
int startIndex = 0;

if (typeIndex > maxChars) {
    startIndex = typeIndex - maxChars;
}

int displayLen = typeIndex - startIndex;

char temp[75];

for (int i = 0; i < displayLen; i++) {
```

```
temp[i] = theText[startIndex + i];  
}
```

Cơ chế này giúp hệ thống hiển thị nội dung theo dạng từng ký tự nhưng không làm mất các ký tự vượt quá khung LED. Nhờ đó, hiệu ứng hiện từng chữ vẫn đảm bảo hiển thị được đầy đủ nội dung, kể cả khi văn bản dài hơn chiều rộng của LED Matrix.

#### 4.3.6. Hiệu ứng nhấp nháy (Blink)

Hệ thống hỗ trợ hiệu ứng nhấp nháy giúp nội dung nổi bật hơn.

```
#Code Flutter  
SwitchListTile(  
  title: const Text(  
    'Hiệu ứng Nhấp Nháy (Blink)',  
    style: TextStyle(fontWeight: FontWeight.bold),  
  ),  
  value: _enableBlink,  
  onChanged: _isEmergencyMode  
    ? null  
    : (val) => setState(() => _enableBlink = val),  
),
```

ESP8266 xử lý trạng thái Blink như sau:

```
String blinkStr = getParam(params, "blink");  
enableBlink = blinkStr == "1";
```

Trong vòng lặp chính, hệ thống thay đổi trạng thái hiển thị theo chu kỳ thời gian.

```
if (enableBlink) {  
  if (millis() - lastBlinkTime > BLINK_INTERVAL) {  
    showText = !showText;  
    lastBlinkTime = millis();  
  }  
}
```

Kết quả là nội dung trên LED Matrix sẽ sáng và tắt luân phiên theo thời gian định sẵn.

#### 4.3.7. Lệnh dừng hiển thị (STOP RUNNING)

Người dùng có thể dừng hoàn toàn việc hiển thị LED Matrix.

```
#Code Flutter
SwitchListTile(
  title: const Text(
    'Dừng hẳn (STOP RUNNING)',
    style: TextStyle(
      fontWeight: FontWeight.bold,
      color: Colors.redAccent,
    ),
  ),
  value: _stopDisplay,
  onChanged: _isEmergencyMode
    ? null
    : (val) => setState(() => _stopDisplay = val),
)
```

ESP8266 xử lý lệnh dừng như sau:

```
if (stopStr == "1")
{
  currentState = STOPPED;
  displayOn = false;
  lmd.clear();
  lmd.display();
}
```

Khi đó, LED Matrix sẽ được xóa toàn bộ nội dung và ngừng cập nhật hiển thị.

#### 4.3.8. Hiển thị thời gian thực

Ứng dụng Flutter hỗ trợ lấy thời gian hiện tại từ điện thoại và gửi xuống LED Matrix.

```
#Code Flutter
final now = DateTime.now();

final String currentTime =
'$${now.hour.toString().padLeft(2, '0')}:${now.minute.toString().padLeft(2, '0')}';
```

Sau khi tạo chuỗi thời gian, ứng dụng gửi xuống ESP8266 bằng hàm `_executeRequest()` để hiển thị lên LED Matrix.

#### 4.3.9. Chức năng xác thực người dùng

Để bảo mật hệ thống người dùng cần phải nhập password để điều khiển LED

Hàm kiểm tra mật khẩu phía Flutter

```
void _checkPassword() {
  if (_isPasswordLocked) {
    _showSnackBar('Bạn đã nhập sai quá nhiều lần. Vui lòng thử lại sau 5 phút.');
```

```

if (_passwordController.text.trim() == _correctPassword) {
  setState(() {
    _isControlUnlocked = true;
    _wrongPasswordCount = 0;
  });
  _showSnackBar('Mở khóa điều khiển thành công!', isSuccess: true);
} else {
  _wrongPasswordCount++;

  if (_wrongPasswordCount >= 3) {
    setState(() {
      _isPasswordLocked = true;
      _isControlUnlocked = false;
    });

    _lockTimer?.cancel();
    _lockTimer = Timer(const Duration(minutes: 5), () {
      if (mounted) {
        setState(() {
          _isPasswordLocked = false;
          _wrongPasswordCount = 0;
        });
      }
    });

    _showSnackBar('Nhập sai 3 lần. Hệ thống bị khóa trong 5 phút.');
```

```

  } else {
    _showSnackBar('Sai mật khẩu. Còn ${3 - _wrongPasswordCount} lần thử.');
```

```

  }
}
}

```

- So sánh mật khẩu người dùng nhập với mật khẩu hệ thống.
- Đếm số lần nhập sai.
- Khóa điều khiển sau 3 lần nhập sai.
- Tự mở khóa sau 5 phút.

Hàm khóa lại quyền điều khiển

```

void _lockControl() {
  setState(() {
    _isControlUnlocked = false;
    _passwordController.clear();
  });
  _showSnackBar('Đã khóa khu vực điều khiển.', isSuccess: true);
}

```

Hàm gửi yêu cầu có kèm mật khẩu

```
String urlString = emergency
?
'http://$ip/?emergency=1&text=$encodedText&speed=${speed.toInt()}&brightness=${brightness.toInt()}&loops=${loops.toInt()}&effect=$effectValue'
:
'http://$ip/?password=$_correctPassword&text=$encodedText&speed=${speed.toInt()}&brightness=${brightness.toInt()}&loops=${loops.toInt()}&effect=$effectValue';
```

→ Sau khi xác thực thành công, mọi lệnh điều khiển đều mang theo mật khẩu để ESP8266 kiểm tra.

Hàm kiểm tra xác thực trên ESP8266

```
// Code Arduino
String passwordStr = getParam(params, "password");
String emergencyStr = getParam(params, "emergency");

// Nếu không phải lệnh khẩn cấp thì bắt buộc đúng password
if (emergencyStr != "1") {
    if (passwordStr != DEVICE_PASSWORD) {
        Serial.println("SAI PASSWORD - KHONG CHO DIEU KHIEN");
        return;
    }
}
```

- Nhận mật khẩu từ ứng dụng Flutter.
- So sánh với mật khẩu lưu trong ESP8266.
- Nếu sai thì từ chối toàn bộ yêu cầu điều khiển.
- Chỉ cho phép bỏ qua kiểm tra trong chế độ khẩn cấp.

#### 4.3.10. Chức năng hẹn giờ tự động

Hệ thống hỗ trợ gửi nội dung tự động theo thời gian đã cài đặt.

```
#Code Flutter
_if (picked != null)
{
    setState(() {
        _selectedTime = picked;
    });
}
```

Sau khi chọn thời gian, ứng dụng tạo bộ đếm Timer:

```
_scheduleTimer = Timer(waitDuration, () async {
    await _executeRequest(
        ip: _scheduledIp!,
        text: _scheduledText!,
        speed: _scheduledSpeed!,
```

```

brightness: _scheduledBrightness!,
loops: _scheduledLoops!,
enableBlink: _scheduledBlink!,
stopDisplay: false,
clearInputAfterSend: false,
);
});

```

Khi đến thời gian hẹn, ứng dụng tự động gửi dữ liệu xuống ESP8266 mà không cần thao tác thủ công.

#### 4.3.11. Chế độ Emergency Mode

Emergency Mode là chế độ cảnh báo khẩn cấp của hệ thống. Khi kích hoạt, LED Matrix sẽ hiển thị nội dung “SOS SOS”, bật nhấp nháy mạnh, tăng độ sáng tối đa và khóa các thành phần điều khiển khác.

```

#Code Flutter
setState(()
{
  _isEmergencyMode = true;
  _enableBlink = true;
  _brightness = 15;
  _speed = 50;
  _loops = 10;
  _stopDisplay = false;
});

```

Sau đó ứng dụng gửi dữ liệu cảnh báo xuống ESP8266:

```

await _executeRequest(
  ip: ip,
  text: 'SOS SOS',
  speed: 50,
  brightness: 15,
  loops: 10,
  enableBlink: true,
  stopDisplay: false,
  clearInputAfterSend: false,
);

```

Khi người dùng tắt Emergency Mode, ứng dụng sẽ gửi lệnh dừng để đưa LED Matrix trở về trạng thái bình thường. Nhờ chế độ này, hệ thống có thể được sử dụng như bảng cảnh báo hoặc tín hiệu khẩn cấp trong các tình huống đặc biệt.

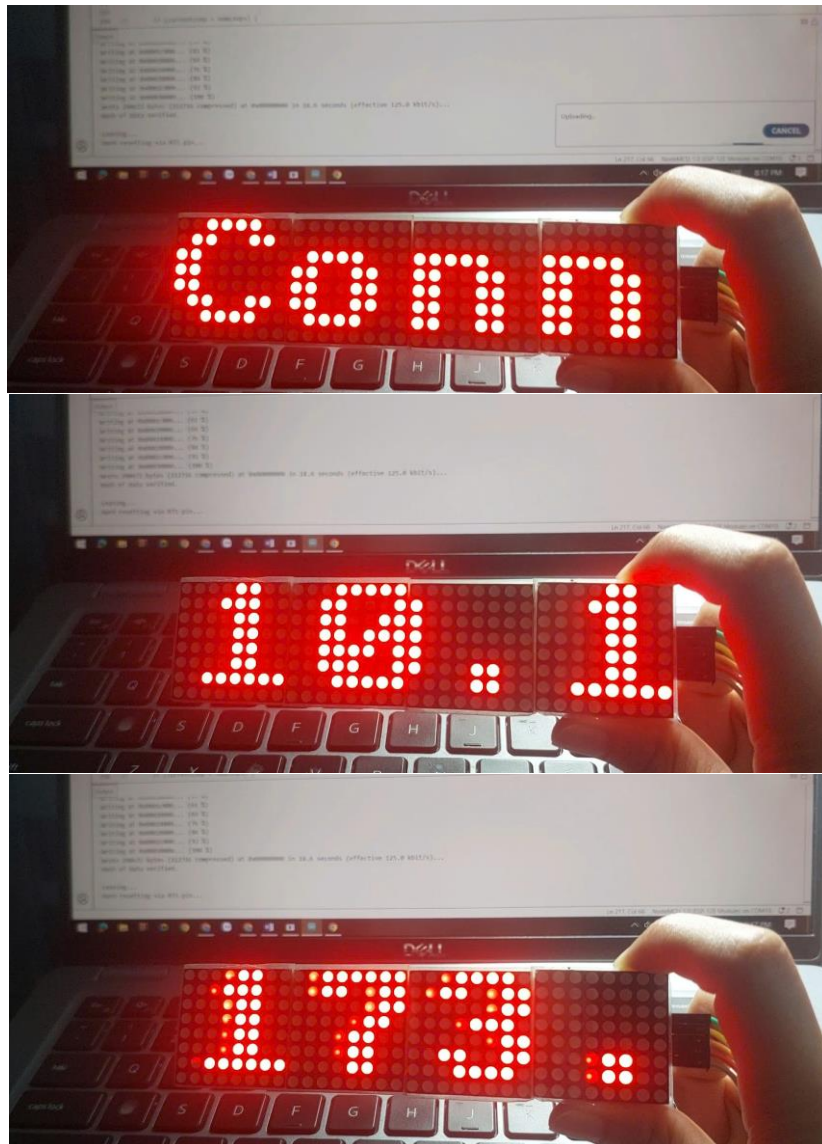
#### 4.4. Triển khai và kiểm thử hệ thống

Sau khi hoàn thiện phần lập trình ESP8266 và ứng dụng Flutter, hệ thống được triển khai thực tế để kiểm tra khả năng hoạt động giữa các thành phần. Quá trình kiểm thử tập trung vào ba nội dung chính: kết nối WiFi, khả năng hiển thị của LED Matrix và hoạt động của ứng dụng Flutter.

#### 4.4.1. Kiểm thử kết nối WiFi

Ở bước này, ESP8266 được nạp chương trình và cấp nguồn để tiến hành kết nối vào mạng WiFi đã cấu hình. Sau khi kết nối thành công, ESP8266 khởi động Web Server trên cổng 80 và hiển thị địa chỉ IP của thiết bị.

Thiết bị di động chạy ứng dụng Flutter được kết nối cùng mạng WiFi với ESP8266. Người dùng nhập địa chỉ IP của ESP8266 vào ứng dụng để thực hiện gửi dữ liệu điều khiển. Nếu địa chỉ IP chính xác và hai thiết bị cùng mạng, ESP8266 sẽ nhận được HTTP Request từ ứng dụng.



Hình 4. Kết nối thành công hiển thị ip kết nối

Kết quả kiểm thử cho thấy ESP8266 có thể kết nối WiFi ổn định, nhận request từ ứng dụng Flutter và phản hồi thành công trong mạng nội bộ.

#### 4.4.2. Kiểm thử hiển thị LED Matrix

Sau khi kết nối thành công, hệ thống tiến hành kiểm thử khả năng hiển thị trên LED Matrix MAX7219. Các nội dung văn bản khác nhau được gửi từ ứng dụng Flutter xuống ESP8266 để đánh giá khả năng hiển thị thực tế.

Các chức năng được kiểm thử gồm: hiển thị văn bản, chạy chữ ngang, hiệu ứng

hiện từng chữ, xử lý văn bản dài vượt khung, điều chỉnh tốc độ, thay đổi độ sáng, thiết lập số vòng lặp, hiệu ứng nhấp nháy và lệnh dừng hiển thị.

<https://drive.google.com/drive/folders/17Gf6WqLCydRmPZJQmnD-iLPhHjxU3T3k?usp=sharing>

Kết quả kiểm thử cho thấy LED Matrix hiển thị đúng nội dung được gửi từ ứng dụng. Các hiệu ứng hoạt động đúng theo thiết lập của người dùng. Đối với nội dung dài hơn kích thước 32x8, hệ thống vẫn hiển thị được đầy đủ thông qua cơ chế chạy chữ hoặc cửa sổ hiển thị từng ký tự.

#### **4.4.3. Kiểm thử ứng dụng Flutter**

Ứng dụng Flutter được kiểm thử trên thiết bị Android để đánh giá khả năng điều khiển hệ thống. Người dùng thực hiện nhập địa chỉ IP, nhập nội dung hiển thị, điều chỉnh tốc độ, độ sáng, số vòng lặp và bật/tắt các hiệu ứng.

Ngoài các chức năng cơ bản, ứng dụng cũng được kiểm thử với chức năng hiển thị giờ hiện tại, hẹn giờ tự động và chế độ Emergency Mode. Khi kích hoạt chế độ khẩn cấp, ứng dụng tự động gửi nội dung cảnh báo, bật nhấp nháy, tăng độ sáng và khóa các thành phần chỉnh sửa.

<https://drive.google.com/drive/folders/17Gf6WqLCydRmPZJQmnD-iLPhHjxU3T3k?usp=sharing>

Kết quả kiểm thử cho thấy giao diện ứng dụng hoạt động ổn định, các nút bấm và thanh điều chỉnh phản hồi đúng chức năng. Khi gửi dữ liệu thành công, LED Matrix cập nhật nội dung tương ứng. Trường hợp nhập thiếu địa chỉ IP, thiếu nội dung hoặc mất kết nối, ứng dụng hiển thị thông báo lỗi để người dùng xử lý.

## CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

### 5.1. Kết luận

Sau quá trình nghiên cứu, thiết kế và triển khai, em đã xây dựng thành công hệ thống điều khiển hiển thị chữ chạy trên LED Matrix 32x8 sử dụng ESP8266 kết hợp với ứng dụng Flutter. Hệ thống cho phép người dùng điều khiển nội dung hiển thị thông qua điện thoại di động bằng kết nối WiFi, giúp việc thao tác trở nên thuận tiện và linh hoạt hơn.

Trong quá trình thực hiện đề tài, em đã tìm hiểu và áp dụng các kiến thức liên quan đến hệ thống nhúng, giao tiếp mạng không dây, lập trình IoT và phát triển ứng dụng di động. ESP8266 được sử dụng để xây dựng Web Server tiếp nhận dữ liệu điều khiển từ ứng dụng Flutter và truyền dữ liệu đến LED Matrix MAX7219 để thực hiện hiển thị nội dung.

Bên cạnh các chức năng cơ bản như hiển thị chữ chạy, điều chỉnh tốc độ, độ sáng và số lần lặp, hệ thống còn được mở rộng thêm nhiều tính năng như hiệu ứng hiển thị từng chữ, chế độ nhấp nháy, hiển thị giờ hiện tại, hẹn giờ tự động và chế độ khẩn cấp Emergency Mode. Các chức năng này giúp tăng tính thực tiễn và khả năng ứng dụng của hệ thống trong nhiều tình huống khác nhau.

Thông qua đề tài, em đã tích lũy thêm kinh nghiệm trong việc thiết kế hệ thống IoT hoàn chỉnh từ phần cứng đến phần mềm, đồng thời nâng cao kỹ năng lập trình ESP8266, phát triển ứng dụng Flutter và xử lý các vấn đề thực tế trong quá trình triển khai hệ thống.

### 5.2. Hướng phát triển

Trong tương lai, hệ thống có thể được mở rộng và nâng cấp theo nhiều hướng nhằm tăng tính ổn định, bảo mật và khả năng ứng dụng thực tế.

Tích hợp các nền tảng như MQTT, Firebase hoặc WebSocket giúp cải thiện khả năng truyền nhận dữ liệu theo thời gian thực và tăng hiệu năng hệ thống.

Phát triển thêm các chức năng quản lý nâng cao như lưu lịch sử nội dung hiển thị, quản lý nhiều bảng LED cùng lúc hoặc hỗ trợ tạo các kịch bản hiển thị tự động theo thời gian.

Mở rộng hệ thống để hỗ trợ hiển thị biểu tượng, hình ảnh đơn giản hoặc hiệu ứng động trên LED Matrix nhằm tăng tính trực quan và khả năng ứng dụng trong thực tế.

Ngoài ra, hệ thống có thể được cải thiện về mặt bảo mật bằng cách sử dụng giao thức HTTPS hoặc bổ sung cơ chế xác thực người dùng nhằm hạn chế truy cập trái phép vào thiết bị ESP8266. Đồng thời, tối ưu giao diện ứng dụng và bổ sung chức năng phản hồi trạng thái trực tiếp từ LED Matrix..

## TÀI LIỆU THAM KHẢO

- [1] <https://docs.flutter.dev/> – Tài liệu chính thức về Flutter framework dùng để phát triển ứng dụng điều khiển LED Matrix.
- [2] <https://github.com/esp8266/arduino> – Mã nguồn và thư viện hỗ trợ ESP8266 cho môi trường Arduino.
- [3] <https://docs.arduino.cc/language-reference/> – Tài liệu tham khảo ngôn ngữ lập trình Arduino.
- [4] <https://www.analog.com/media/en/technical-documentation/data-sheets/max7219-max7221.pdf> – Thông tin kỹ thuật và nguyên lý hoạt động của MAX7219.
- [7] <https://docs.arduino.cc/libraries/ledmatrixdriver/> – Thông tin thư viện LEDMatrixDriver cho Arduino và ESP8266.